

PL SQL 1 – 22AIE303

Roll No : AM.SC.U4AIE23140

Name : P Arya

QN: 1

Dashboard Properties SQL Statistics Dependencies Dependents postgres/postgres@PostgreSQL 14

postgres/postgres@PostgreSQL 14

Query Editor Query History

```
1 --a function to accept two integers and returns their sum
2 create or replace function intsum(int, int)
3     returns int as
4 $$
5     Begin
6         return $1+$2;
7     End;
8 $$
9 Language 'plpgsql';
10 select intsum(5,9);
11
```

Data Output Explain Messages Notifications

	intsum	
	integer	
1	14	

postgres/postgres@PostgreSQL 14

Query Editor Query History

```
1 --function that accepts an integer and returns whether its odd or even
2 create or replace function oddeven(int)
3     returns varchar as
4 $$
5     Begin
6         if $1%2=0 then
7             return 'even';
8         else
9             return 'odd';
10        end if;
11    End;
12 $$
13 Language 'plpgsql';
14 select oddeven(5);
15
```

Data Output Explain Messages Notifications

	oddeven	
	character varying	
1	odd	

postgres/postgres@PostgreSQL 14

Query Editor Query History

```

1 --function which accepts employee number and returns salary of that employee
2 create or replace function emp_sal(empno Emp.eno%type)
3   returns Emp.sal%type as
4 $$
5   Declare
6     salary Emp.sal%type;
7   Begin
8     select sal into salary from Emp where eno=empno;
9     return salary;
10  End;
11 $$
12 Language 'plpgsql';
13 select emp_sal(115);
14

```

Data Output Explain Messages Notifications

	emp_sal	numeric
1		66000.00

postgres/postgres@PostgreSQL 14

Query Editor Query History

```

1 --function that accepts a department number and give the no of employees in that department
2 create or replace function dno_noofemp(deptno Emp.dno%type)
3   returns int as
4 $$
5   Declare
6     num int;
7   Begin
8     select count(eno) into num from Emp where dno=deptno;
9     return num;
10  End;
11 $$
12 Language 'plpgsql';
13 select dno_noofemp(102);
14

```

Data Output Explain Messages Notifications

	dno_noofemp	integer
1		3

postgres/postgres@PostgreSQL 14

Query Editor Query History

```

1 --function that takes employee number and give employee 10% salary in hike if avg salary of that dpt is great
2 create or replace function sal_hike(empNo Emp.eno%type)
3   returns varchar as
4 $$
5   Declare
6     salary Emp.sal%type;
7     avgSal Emp.sal%type;
8   Begin
9     select sal into salary from Emp where eno=empNo;
10    select avg(sal) into avgSal from Emp where dno=(select dno from Emp where eno=empNo);
11    if (salary< avgSal) then
12      update Emp set sal=sal+sal*0.1 where eno=empNo;
13      return 'Salary updated';
14    else
15      return 'no update';
16    end if;
17  End;
18 $$
19 Language 'plpgsql';
20 select sal_hike(115);
21

```

Data Output Explain Messages Notifications

	sal_hike	character varying
1		no update

postgres/postgres@PostgreSQL 14

Query Editor Query History

```

1  --function that accepts an employee name and returns dept name of employee
2  create or replace function name_dept(empname Emp.ename%type)
3      returns varchar as
4  $$
5      Declare
6          depname Deptt.dname%type;
7      Begin
8          select dname into depname from Deptt where dept_no=(select dno from Emp where ename=empname);
9          return depname;
10     End;
11 $$
12 Language 'plpgsql';
13 select name_dept('Thor');
14
15
16

```

Data Output Explain Messages Notifications

name_dept
character varying

1	cs
---	----

QN : 2

postgres/postgres@PostgreSQL 17

Query Query History

```

1  --Qn2
2  --a. function ProjectLoad that returns the total project working hours for the given eno.
3  create or replace function projectload(empno emp.eno%type) returns numeric as
4  $$
5  Declare
6      total_hrs numeric := 0;
7  Begin
8      select sum(prjt_hrs) into total_hrs from emp_proj where empno=eno;
9      return total_hrs;
10 End;
11 $$
12 Language 'plpgsql';
13 select projectload(102);
14

```

Data Output Messages Notifications

projectload
numeric

1	55.5
---	------

Showing rows:

postgres/postgres@PostgreSQL 17

Query Query History

```

16 --Qn2
17 --b. function to update the salary of an employee if the employee earn less than the average salary.(new sal = curr sal + diff in avg and curr)
18 create or replace function sal_hike(empno emp.eno%type) returns varchar as
19 $$
20 Declare
21     currsal emp.sal%type;
22     avgsal emp.sal%type;
23 Begin
24     select sal into currsal from emp where eno=empno;
25     select avg(sal) into avgsal from emp;
26     if currsal<avgsal then
27         update emp set sal=sal+(avgsal-currsal) where eno=empno;
28         return 'salary updated';
29     else
30         return 'no salary update';
31     end if;
32 End;
33 $$
34 Language 'plpgsql';
35 select sal_hike(105)
36

```

Data Output Messages Notifications

sal_hike
character varying

1	salary updated
---	----------------

Showing rows: 1 to 1 Page No: 1 of 1

QN : 3

Query History

```
--Qn 3
-- function to accept an item no from the user. If transaction has been made for less than 2 times for that item, delete the item from the Item
create or replace function delete_rare(inumber item.ino%type) returns varchar as
$$
Declare
trans_count int :=0;
Begin
select count(tr_no) into trans_count from transactions where ino=inumber;
if trans_count<2 then
delete from item where ino=inumber;
return 'rare item removed';
else
return 'item not removed';
end if;
End;
$$
Language 'plpgsql';
select delete_rare(1002);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

delete_rare
1 rare item removed

QN : 4

Query History

```
--qn 4
--a. a function that accepts a threshold value and a customer number .The program updates the c_type based on the threshold value. Ie. If balance>
create or replace function class_update(threshold int,cust_number customer.cno%type) returns varchar as
$$
Declare
Tot_bal int;
Begin
select sum(bal) into Tot_bal from accounts where cust_no=cust_number;
if Tot_bal>threshold then
update customer set c_type='a' where cno=cust_number;
return 'type updated';
else
update customer set c_type='b' where cno=cust_number;
return 'no type updation';
end if;
End;
$$
Language 'plpgsql';
select class_update(4500,3);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

class_update
1 type updated

Query History

```
--Qn 4
--b. a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all
create or replace function CloseBranch(close_br branches.br_no%type , new_br branches.br_no%type) returns varchar as
$$
Declare
br_cnt int :=0;
Begin
select count(*) into br_cnt from branches where br_no =close_br;
if br_cnt = 0 then
return 'branch to close does not exist';
end if;
select count(*) into br_cnt from branches where br_no =new_br;
if br_cnt = 0 then
return 'target branch does not exist';
end if;
update accounts set br_no =new_br where br_no =close_br;
delete from branches where br_no = close_br;
return 'branches moved';
End;
$$
Language 'plpgsql';
select CloseBranch(10,20);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

closebranch
1 branches moved

✓ Successfully run. Total query runtime: 120 msec. 1 rows affected. ✕

postgres/postgres@PostgreSQL 17

Query Query History

```
--Qn 4
--c. a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.
create or replace function safewithdrawal(accNumber accounts.ac_no%type, withdrawAmt accounts.bal%type) returns varchar as
$$
Declare
    BalAmt accounts.bal%type;
Begin
    select bal into BalAmt from accounts where ac_no = accNumber;
    if withdrawAmt <= BalAmt then
        update accounts set bal=bal-withdrawAmt where ac_no =accNumber;
        return 'withdrawal successful';
    else
        return 'insufficient fund';
    end if;
End;
$$
Language 'plpgsql';
select safewithdrawal(1006,54000),safewithdrawal(1006,87000) ;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	safewithdrawal character varying	safewithdrawal character varying
1	withdrawal successful	insufficient fund